



OLD DOMINION
UNIVERSITY



CCL Group Training

2026-2027 HPC & Linux Systems

Date 09/18/2026



RECAP (last training)

- **Available ODU Clusters – Wahab, etc.**
- **SSH Terminal Connection – mobaXterm (Windows), etc.**
- **Basic Shell Commands**
- **VI editor**



Training Overview

- **Environment Setup (.bashrc)**
- **SSH file transfer or SCP utility**
- **CFD software installation – Converge bundle, OpenFOAM**
- **Tutorials for CONVERGE simulations (license required)**
- **Conda setup on Wahab**
- **Job management**



.bashrc

`.bashrc` is a shell script executed every time a user starts a new interactive non-login shell (e.g., opening a terminal window or running a subshell). Located in your home directory at `~/.bashrc`, it configures your individual bash command-line environment.

- **File Location:** `~/.bashrc` (`/home/username/.bashrc`)
- **Execution Trigger:** Opening an interactive non-login bash terminal
- **Hidden File:** Standard `ls` hides it because of the leading dot; view with `ls -a ~/.bashrc`
- **Manual Reloading:** Changes take effect in new terminals, or instantly in the current terminal via `source ~/.bashrc`

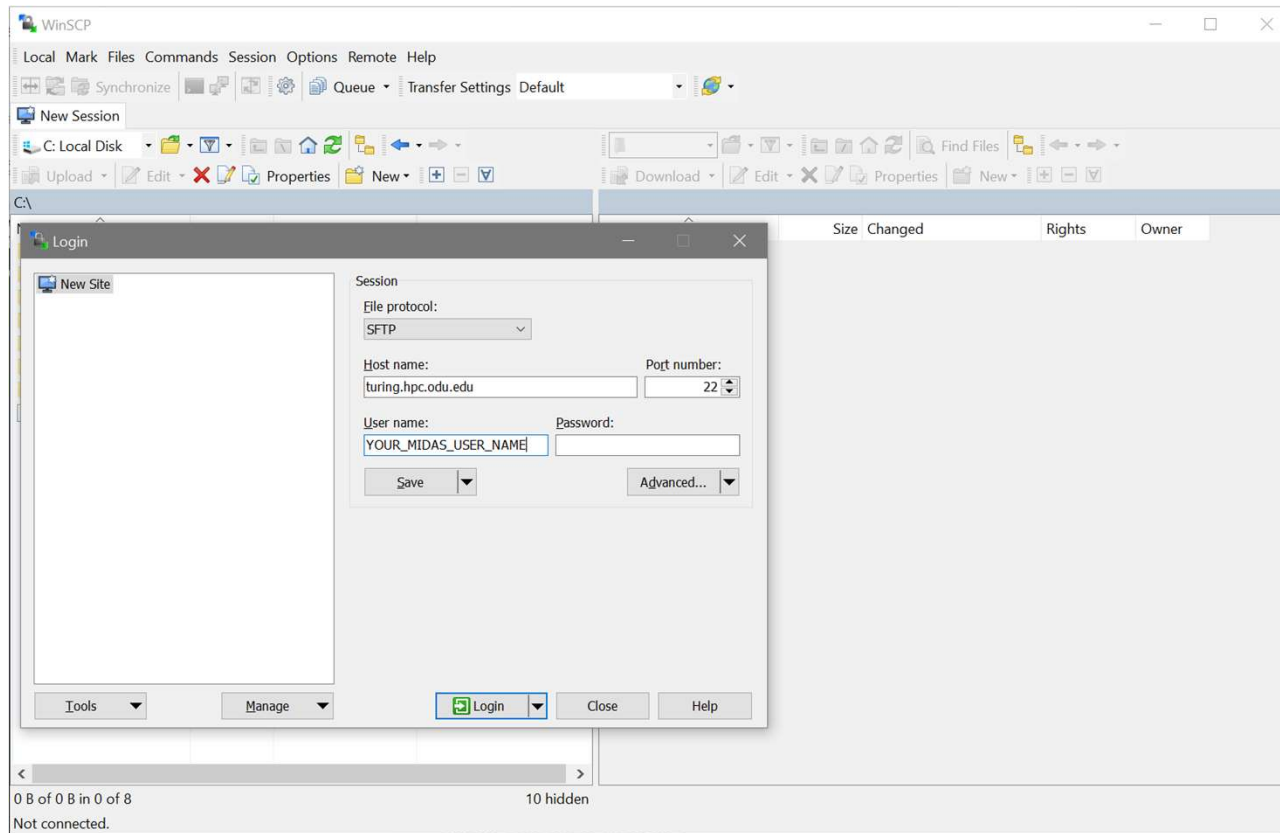
Core Uses

- **Environment Variables:** Export systems paths or settings used by programs (e.g., `export PATH="$HOME/bin:$PATH"` or `export RLM_LICENSE=27000@server`).
- **Aliases:** Create shortcuts for long or frequently used commands (e.g., `alias ll='ls -la'`).
- **Module & Tool Auto-loading:** Automatically run scripts, load modules, or initialize tools (like Conda or Spack) upon login.



File Transfer

SCP/SFTP for Windows (<https://winscp.net/eng/index.php>)





File Transfer

SCP command for macOS/Linux/Windows:

- Copy data from local to remote server:

```
scp file MIDAS_USERNAME@wahab.hpc.odu.edu:/path/to/designated/location
```

- Copy data from remote server to local:

```
scp MIDAS_USERNAME@wahab.hpc.odu.edu:/path/to/file /path/local/directory
```

- When copying a directory, add `-r`:

```
scp -r dor MIDAS_USERNAME@wahab.hpc.odu.edu:/path/to/designated/location  
scp MIDAS_USERNAME@wahab.hpc.odu.edu:/path/to/dir /path/local/directory
```

See <https://wiki.hpc.odu.edu/en/FileTransfer> for more details



CONVERGE bundle installation

Prerequisites

- A valid file (license.lic)
- Downloaded installation file (CONVERGE_CFD_Bundle_<version>_linux64.sh) from the **Convergent Science Hub**.
 - Transfer the downloaded .sh file to your cluster directory (e.g., /home/username/downloads or a shared group folder /apps/software) using scp or rsync.
 - Execute the interactive installer:
`./CONVERGE_CFD_Bundle_*.sh`
 - Configure Environment Variables & License in your shell configuration (~/.bashrc or ~/.bash_profile):
`export RLM_LICENSE=2765@converge.license.odu.edu`
 - Load the bundle environment script created by the installer:
`source /path/to/installation/Convergent_Science/converge_env.sh`
 - Verify Installation:
`Converge-inlet --version`



CONVERGE bundle installation (continued)

Running Headless vis Slurm Batch Scripts: Do NOT run compute jobs directly on the head (login) node. Submit a batch script to your job scheduler (e.g., Slurm) using **mpirun** or **sun** (See ODU RCC Wiki)

```
#!/bin/bash#SBATCH
--job-name=converge_sim
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=40
#SBATCH --time=12:00:00
```

```
# Load environment
source /path/to/installation/Convergent_Science/converge_env.sh
```

```
# Run executable in your case directory
mpirun converge-intel --super > run.log 2>&1
```




OpenFOAM (spack installation)

Installing OpenFOAM without root privileges is standard practice on HPC clusters or shared linux environments. The two best methods are building from source using **Spack** or compiling manually into your user home directory.

Method 1: Installing via Spack

- Clone the Spack repository into your home directory:
`git clone -c feature.manyFiles=true https://github.com/spack/spack.git ~/spack`
- Add Spack to your current shell session:
`~/spack/share/spack/setup-env.sh`
- Install OpenFOAM (Spack will automatically download and build dependencies like MPI and CMake locally):
`spack install openfoam`
- Load the newly installed OpenFOAM module:
`Spack load openfoam`



Conda Module on Wahab

On the Wahab HPC cluster at Old Dominion University (ODU), software management uses containerized environments (`container_env`) for isolated python execution rather than directly installing raw standard packages.

Commands must be routed using ODU's `crun` wrapper to ensure binaries execute inside the containerized file system.

Step 1: Load the Container Module

- Run the following command once per terminal session (or you could add it in `.bashrc`):
`module load container_env python3`

Step 2: Create a Dedicated Conda Environment for Cantera

- Create the target directory structure:
`mkdir -p ~/envs`
- Initialize the environment using `crun` and `conda-forge`:
`crun -c -s -p ~/envs/ct-env conda create -c conda-forge -y python=3.9 cantera matplotlib ipython`

`-c -s`: Flags that inform `crun` to initialize and setup the new container workspace.

`-p ~/envs/ct-env`: Specifies the path where the environment resides.

`-c conda-forge`: Fetches `cantera` and its dependencies from the official `conda-forge` channel.



Conda Module on Wahab (continued)

Step 3: Run Cantera Scripts or Interactively Test

Instead of standard `conda activate`, Wahab HPC executes commands within the environment by prefixing them with `crun -p <PATH>`:

- Test Python import:

```
crun -p ~/envs/ct-env python3 -c "import cantera as ct; print(ct.__version__)"
```

- Run a custom script (test.py):

```
crun -p ~/envs/ct-env python 3 test.py
```



Conda Module on Wahab (continued)

Step 4: Configure for Slurm Batch Jobs

When submitting production jobs on Wahab, do **NOT** run jobs on login nodes. Include the container environment inside your Slurm script:

```
#!/bin/bash#SBATCH
--job-name=converge_sim
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=40
#SBATCH --time=12:00:00

# Load environment
module load container_env python3

# Run executable in your case directory
crun -p ~/envs/ct-env python3 your_utility_script.py
```